

Week 7.2 Exercise

May 2, 2026

1 Rebuild: GPT-2 Fine-Tuning with LoRA (MacFarquhar Article)

This notebook rebuilds the workflow from Matthew MacFarquhar's post *Sculpting Language: GPT-2 Fine-Tuning with LoRA*.

Article used: <https://blog.devgenius.io/sculpting-language-gpt-2-fine-tuning-with-lora-1caf3bfbc3c6>

My goal in this assignment is not just to copy the code, but to explain what each step does, why it is done, and how well the approach works in practice.

1.1 1. Colab Setup (GPU Required)

The author notes that training benefits from a GPU. For this assignment, use Google Colab and set:
- Runtime -> Change runtime type -> Hardware accelerator: GPU - GPU type: T4 (if available)

Why this matters: LoRA is parameter-efficient, but we still run backprop through GPT-2 layers. A T4 dramatically reduces training time compared with CPU.

```
[1]: # If running in Colab, install dependencies
!pip install -q transformers datasets peft accelerate
```

```
[2]: import torch

print('Torch:', torch.__version__)
print('CUDA available:', torch.cuda.is_available())
if torch.cuda.is_available():
    print('GPU:', torch.cuda.get_device_name(0))
```

```
Torch: 2.11.0
CUDA available: False
```

1.2 2. Imports and Core Configuration

In the article, the workflow is simple: load GPT-2, freeze base weights, add LoRA adapters, and train on a transformed quote-tag dataset.

I keep the same spirit but make the notebook slightly more explicit with named constants so it is easier to tune and explain.

```
[3]: from datasets import load_dataset
from peft import LoraConfig, get_peft_model, PeftModel
from transformers import (
    AutoModelForCausalLM,
    AutoTokenizer,
    DataCollatorForLanguageModeling,
    Trainer,
    TrainingArguments,
)

MODEL_NAME = 'gpt2'
OUTPUT_DIR = './macfarquhar_lora_gpt2'
MAX_LENGTH = 128
DEVICE = 'cuda' if torch.cuda.is_available() else 'cpu'

print('Device:', DEVICE)
```

Device: cpu

1.3 3. Quick Baseline Before Fine-Tuning

The blog first shows GPT-2 does poorly for the quote-tagging format.

Task format: “quote ->: tags”

My interpretation: this baseline is important because it proves the model does not already solve the task out of the box.

```
[4]: baseline_model = AutoModelForCausalLM.from_pretrained(MODEL_NAME).to(DEVICE)
baseline_tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
baseline_tokenizer.pad_token = baseline_tokenizer.eos_token

test_prompt = '"Life is like a box of chocolates, you never know what you are_
↳gonna get" ->:'
inputs = baseline_tokenizer(test_prompt, return_tensors='pt').to(DEVICE)

with torch.no_grad():
    out = baseline_model.generate(
        **inputs,
        max_new_tokens=30,
        do_sample=True,
        top_p=0.95,
        temperature=0.8,
        pad_token_id=baseline_tokenizer.eos_token_id,
    )

print('Baseline output:')
print(baseline_tokenizer.decode(out[0], skip_special_tokens=True))
```

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.

```
config.json: 0%|          | 0.00/665 [00:00<?, ?B/s]
model.safetensors: 0%|          | 0.00/548M [00:00<?, ?B/s]
Loading weights: 0%|          | 0/148 [00:00<?, ?it/s]
generation_config.json: 0%|          | 0.00/124 [00:00<?, ?B/s]
tokenizer_config.json: 0%|          | 0.00/26.0 [00:00<?, ?B/s]
vocab.json: 0.00B [00:00, ?B/s]
merges.txt: 0.00B [00:00, ?B/s]
tokenizer.json: 0.00B [00:00, ?B/s]
```

Baseline output:

```
"Life is like a box of chocolates, you never know what you are gonna get" ->:
"It will be perfect" [1]
```

(and that's not even talking about the two videos, where you can see a more realistic

1.4 4. Build the LoRA Training Model

What the author is doing here: 1. Load GPT-2 and tokenizer. 2. Freeze original GPT-2 parameters. 3. Add LoRA adapters (rank $r=16$, alpha 32, dropout 0.05).

My commentary: this is the central efficiency idea. Instead of updating all GPT-2 weights, we only train a compact adapter. This reduces memory and compute while preserving most of GPT-2's pre-trained language knowledge.

```
[5]: model = AutoModelForCausalLM.from_pretrained(MODEL_NAME, device_map='auto')
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
tokenizer.pad_token = tokenizer.eos_token

# Freeze base model weights
for p in model.parameters():
    p.requires_grad = False

lora_config = LoraConfig(
    r=16,
    lora_alpha=32,
    lora_dropout=0.05,
    bias='none',
    task_type='CAUSAL_LM',
    target_modules=['c_attn'],
)
```

```

model = get_peft_model(model, lora_config)
model.config.use_cache = False

trainable = sum(p.numel() for p in model.parameters() if p.requires_grad)
total = sum(p.numel() for p in model.parameters())
print(f'Trainable params: {trainable:,}')
print(f'Total params: {total:,}')
print(f'Trainable percentage: {100 * trainable / total:.4f}%')

```

Loading weights: 0% | 0/148 [00:00<?, ?it/s]

Trainable params: 589,824

Total params: 125,029,632

Trainable percentage: 0.4717%

```

/opt/anaconda3/lib/python3.12/site-packages/peft/tuners/lora/layer.py:2504:
UserWarning: fan_in_fan_out is set to False but the target module is `Conv1D`.
Setting fan_in_fan_out to True.
warnings.warn(

```

1.5 5. Load and Transform the Dataset

The blog uses Hugging Face dataset `Abirate/english_quotes`.

The key transformation is to merge columns into one causal-language target text: - input text: `"{quote}" ->: {tags}`

My evaluation: the `->:` delimiter is a simple but effective supervision signal. It explicitly marks where the model should switch from quote text to tag generation.

```

[6]: dataset = load_dataset('Abirate/english_quotes')
print(dataset)
print(dataset['train'][0])

def merge_columns(example):
    quote = str(example['quote']).strip().replace('\n', ' ')
    tags = str(example['tags']).strip()
    example['prediction'] = f'"{quote}" ->: {tags}'
    return example

dataset = dataset.map(merge_columns)
dataset = dataset['train'].train_test_split(test_size=0.1, seed=42)
print(dataset)

```

README.md: 0.00B [00:00, ?B/s]

quotes.jsonl: 0% | 0.00/647k [00:00<?, ?B/s]

Generating train split: 0% | 0/2508 [00:00<?, ? examples/s]

```

DatasetDict({
  train: Dataset({

```

```

        features: ['quote', 'author', 'tags'],
        num_rows: 2508
    })
})
{'quote': '"Be yourself; everyone else is already taken."', 'author': 'Oscar
Wilde', 'tags': ['be-yourself', 'gilbert-perreira', 'honesty', 'inspirational',
'misattributed-oscar-wilde', 'quote-investigator']}
Map:   0%|          | 0/2508 [00:00<?, ? examples/s]
DatasetDict({
  train: Dataset({
    features: ['quote', 'author', 'tags', 'prediction'],
    num_rows: 2257
  })
  test: Dataset({
    features: ['quote', 'author', 'tags', 'prediction'],
    num_rows: 251
  })
})

```

1.6 6. Tokenization and Training Setup

The article trains with warmup and fixed training steps. I keep that structure: - warmup steps: 100 - max steps: 500 - effective batch size: $4 \times 4 = 16$ (via gradient accumulation)

My commentary: for a small dataset, this schedule is enough to show learning, but it can overfit. I include a validation split to observe generalization behavior during training.

```

[8]: def tokenize_function(batch):
    tokens = tokenizer(
        batch['prediction'],
        truncation=True,
        padding='max_length',
        max_length=MAX_LENGTH,
    )
    tokens['labels'] = tokens['input_ids'].copy()
    return tokens

tokenized = dataset.map(tokenize_function, batched=True)
tokenized = tokenized.remove_columns(['quote', 'author', 'tags', 'prediction'])

data_collator = DataCollatorForLanguageModeling(tokenizer=tokenizer, mlm=False)

training_args = TrainingArguments(
    output_dir=OUTPUT_DIR,
    per_device_train_batch_size=4,
    gradient_accumulation_steps=4,
    learning_rate=2e-4,

```

```

warmup_steps=100,
max_steps=500,
logging_steps=25,
eval_steps=100,
save_steps=100,
eval_strategy='steps', # Changed from 'evaluation_strategy' to
↪ 'eval_strategy'
save_total_limit=2,
fp16=torch.cuda.is_available(),
report_to='none',
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized['train'],
    eval_dataset=tokenized['test'],
    data_collator=data_collator,
)

```

1.7 7. Train and Save LoRA Weights

This is the main training execution.

My commentary: saving adapters with `save_pretrained` is cleaner than a raw `state_dict` file because it keeps LoRA config + weights together and is easier to reload.

```

[9]: train_result = trainer.train()
print(train_result)

model.save_pretrained(OUTPUT_DIR)
tokenizer.save_pretrained(OUTPUT_DIR)
print('Saved LoRA adapter + tokenizer to', OUTPUT_DIR)

```

```

/opt/anaconda3/lib/python3.12/site-packages/torch/utils/data/dataloader.py:775:
UserWarning: 'pin_memory' argument is set as true but not supported on MPS now,
device pinned memory won't be used.

```

```

    super().__init__(loader)
[transformers] `loss_type=None` was set in the config but it is unrecognized.
Using the default loss: `ForCausalLMLoss`.

```

```

<IPython.core.display.HTML object>

```

```

/opt/anaconda3/lib/python3.12/site-packages/torch/utils/data/dataloader.py:775:
UserWarning: 'pin_memory' argument is set as true but not supported on MPS now,
device pinned memory won't be used.

```

```

    super().__init__(loader)
/opt/anaconda3/lib/python3.12/site-packages/torch/utils/data/dataloader.py:775:
UserWarning: 'pin_memory' argument is set as true but not supported on MPS now,

```

```

device pinned memory won't be used.
    super().__init__(loader)
/opt/anaconda3/lib/python3.12/site-packages/torch/utils/data/dataloader.py:775:
UserWarning: 'pin_memory' argument is set as true but not supported on MPS now,
device pinned memory won't be used.
    super().__init__(loader)
/opt/anaconda3/lib/python3.12/site-packages/torch/utils/data/dataloader.py:775:
UserWarning: 'pin_memory' argument is set as true but not supported on MPS now,
device pinned memory won't be used.
    super().__init__(loader)

TrainOutput(global_step=500, training_loss=2.9554358825683593,
metrics={'train_runtime': 1531.675, 'train_samples_per_second': 5.223,
'train_steps_per_second': 0.326, 'total_flos': 523248022978560.0, 'train_loss':
2.9554358825683593, 'epoch': 3.5238938053097346})
Saved LoRA adapter + tokenizer to ./macfarquhar_lora_gpt2

```

1.8 8. Inference with the Fine-Tuned Adapter

This mirrors the article's second stage (inference). We rebuild the base GPT-2 model and attach the saved LoRA adapter before generation.

```

[10]: base_for_infer = AutoModelForCausalLM.from_pretrained(MODEL_NAME).to(DEVICE)
fine_tuned_model = PeftModel.from_pretrained(base_for_infer, OUTPUT_DIR).
    ↳to(DEVICE)
infer_tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
infer_tokenizer.pad_token = infer_tokenizer.eos_token

test_prompt = '"Life is like a box of chocolates, you never know what you are_
    ↳gonna get" ->:'
inputs = infer_tokenizer(test_prompt, return_tensors='pt').to(DEVICE)

with torch.no_grad():
    out = fine_tuned_model.generate(
        **inputs,
        max_new_tokens=30,
        do_sample=True,
        top_p=0.95,
        temperature=0.7,
        pad_token_id=infer_tokenizer.eos_token_id,
    )

print('Fine-tuned output:')
print(infer_tokenizer.decode(out[0], skip_special_tokens=True))

```

```
Loading weights: 0%|          | 0/148 [00:00<?, ?it/s]
```

Fine-tuned output:

```
"Life is like a box of chocolates, you never know what you are gonna get" ->:
```

```
['chocolate', 'life', 'love'] ['candy', 'love'] ['chocolate'] ['life-in-the-box',
```

1.9 9. Base GPT-2 vs LoRA: Multi-Prompt Comparison

This section adds a direct side-by-side comparison over multiple unseen quotes.

My evaluation approach: - We run the same prompt format through both models. - We inspect qualitative differences in output relevance. - We compute a small heuristic score to summarize whether output looks tag-like (not a formal benchmark).

This is not a substitute for strict metrics, but it gives a quick sanity check for assignment purposes.

```
[11]: import re
import pandas as pd

comparison_quotes = [
    "Life is like a box of chocolates, you never know what you are gonna get",
    "Be yourself; everyone else is already taken",
    "In the middle of difficulty lies opportunity",
    "Do what you can, with what you have, where you are",
    "Success is not final, failure is not fatal",
    "The best way out is always through",
    "Happiness depends upon ourselves",
    "Turn your wounds into wisdom",
]

def generate_completion(model_obj, tok, quote, max_new_tokens=20, temperature=0.
↪7):
    prompt = f'"{quote}" ->:'
    encoded = tok(prompt, return_tensors='pt').to(model_obj.device)
    with torch.no_grad():
        out = model_obj.generate(
            **encoded,
            max_new_tokens=max_new_tokens,
            do_sample=True,
            top_p=0.95,
            temperature=temperature,
            pad_token_id=tok.eos_token_id,
        )
    decoded = tok.decode(out[0], skip_special_tokens=True)
    generated = decoded[len(prompt):].strip()
    return generated

def heuristic_score(text):
    # Lightweight proxy score for tag-like output
    cleaned = text.strip().lower()
    non_empty = int(len(cleaned) > 0)
    concise = int(1 <= len(cleaned.split()) <= 12)
```



```

tag_like = int(bool(re.search(r",|-|\b[a-z]{3,}\b", cleaned)))
return (non_empty + concise + tag_like) / 3.0

rows = []

for quote in comparison_quotes:
    baseline_pred = generate_completion(baseline_model, baseline_tokenizer,
    ↪quote, temperature=0.8)
    lora_pred = generate_completion(fine_tuned_model, infer_tokenizer, quote,
    ↪temperature=0.7)

    rows.append(
        {
            "quote": quote,
            "baseline_output": baseline_pred,
            "lora_output": lora_pred,
            "baseline_score": round(heuristic_score(baseline_pred), 3),
            "lora_score": round(heuristic_score(lora_pred), 3),
        }
    )

comparison_df = pd.DataFrame(rows)
comparison_df["score_delta"] = (comparison_df["lora_score"] -
    ↪comparison_df["baseline_score"]).round(3)

print("Average baseline score:", round(comparison_df["baseline_score"].mean(),
    ↪3))
print("Average LoRA score:", round(comparison_df["lora_score"].mean(), 3))
print("Average delta (LoRA - baseline):", round(comparison_df["score_delta"].
    ↪mean(), 3))

comparison_df

```

Average baseline score: 0.709
 Average LoRA score: 1.0
 Average delta (LoRA - baseline): 0.291

```

[11]:                                     quote \
0  Life is like a box of chocolates, you never kn...
1      Be yourself; everyone else is already taken
2      In the middle of difficulty lies opportunity
3  Do what you can, with what you have, where you...
4      Success is not final, failure is not fatal
5      The best way out is always through
6      Happiness depends upon ourselves
7      Turn your wounds into wisdom

```

	baseline_output \		lora_output	baseline_score \
0	"Hey, there's something I want to do" ->: "How...		['life', 'love', 'life'] ['life-of-love'] ['li...	0.667
1	"I can't see my friends anymore" ->: "I'll nev...		['self', 'being', 'being-self', 'self-love'] -...	0.667
2	"In the middle of difficulty lies opportunity"...		['desire', 'inspiration'] -> ['inspiration', '...	0.667
3	"You can do what you want, but you'll always h...		['help', 'help-yourself', 'help-yourself', 'he...	0.667
4	"Success is not final, failure is not fatal" -...		['success', 'failure'] ['success', 'failure'] ...	0.667
5	["Habits"] -> ["Ladders"] -> ["Climbing", "Sta...		['self-help', 'self-help'] ['self-help'] ['sel...	1.000
6	"I want to feel good and want to be happy." \n \n		['happiness', 'happiness-love'] ['love'] ['lov...	0.667
7	"Solve all of your conflicts" \n \n You can find ...		['haunted,' 'soul,' 'soul-reading'] ['wisdom']...	0.667

	lora_score	score_delta
0	1.0	0.333
1	1.0	0.333
2	1.0	0.333
3	1.0	0.333
4	1.0	0.333
5	1.0	0.000
6	1.0	0.333
7	1.0	0.333

1.10 9. My Evaluation of the Author's Approach

1.10.1 What works well

- The pipeline is easy to reproduce and teaches the core LoRA idea clearly.
- GPT-2 is small enough for students to train with modest GPU access.
- The input reformulation (quote ->: tags) is simple and practical for causal LM fine-tuning.

1.10.2 Limitations I observed

- The task is narrow, and the dataset is relatively small, so outputs can become repetitive.
- The generated tags are often plausible but not always cleanly formatted or semantically precise.
- More rigorous evaluation metrics (exact tag match, F1, or semantic overlap) are needed for strong claims.

1.10.3 My conclusion

For a course assignment, this is a strong demonstration of parameter-efficient fine-tuning. It shows that a smaller model with LoRA can learn a focused formatting task quickly on a T4 GPU. However, production-quality performance would require broader data, stricter evaluation metrics, and additional prompt/output controls.